



Research paper

Federated continual representation learning for evolutionary distributed intrusion detection in Industrial Internet of Things

Zhao Zhang^a, Yong Zhang^b, Hao Li^c, Shenbo Liu^a, Wei Chen^a, Zhigang Zhang^a, Lijun Tang^{a,*}

^a School of Physics and Electronic Science, Changsha University of Science and Technology, Changsha, 410114, China

^b School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, 100876, China

^c School of Traffic and Transportation Engineering, Changsha University of Science and Technology, 410114, Changsha, China

ARTICLE INFO

Keywords:

Distributed intrusion detection
Federated learning
Representation learning
Industrial Internet of Things

ABSTRACT

As a promising paradigm, Federated Learning (FL)-based distributed intrusion detection offers potent protection for the network security of Industrial Internet of Things (IIoT) systems. Nonetheless, the practical deployment of IIoT systems occurs in a highly-complex and dynamic distributed environment. The ever-growing and dynamically-evolving cyber attacks will render the FL-based intrusion detection model inefficient, since FL cannot gracefully learn from the dynamic traffic data streams to identify new attacks. To address this issue, we propose the evolutionary distributed intrusion detection system based on federated continual representation learning, designed to continually capture effective feature representations of emerging attacks from dynamic traffic data streams. Specifically, we develop the supervised contrastive loss and the global information-aware regularization loss to alleviate the catastrophic forgetting on the previously observed attacks and mitigate the data heterogeneity across clients. Besides, we propose the prototype variance-based memory update strategy to ensure the effective memory replay data. Extensive experimental results demonstrate our proposed method outperforms the state-of-the-art methods by 13.3%–31.5% in terms of average accuracy on a real energy intrusion detection dataset.

1. Introduction

The advent of cutting-edge information technologies and the proliferation of smart devices have facilitated the seamless integration of physical and cyber components in industrial processes, giving birth to the Industrial Internet of Things (IIoT) (Chi et al., 2022; Wu et al., 2021). IIoT embodies a networked interconnection of high-precision sensors, quick-response actuators, and automated devices, with the shared goal of amplifying reliability and operational efficiency. Despite the remarkable advances in efficiency and reliability, IIoT still struggles with the potential security loopholes (Salim et al., 2024), which will inevitably trigger a certain amount of information security issues. Consequently, the inherent decentralization and vulnerability of IIoT-enabled systems necessitate the urgent deployment of distributed network intrusion detection systems to counteract and prevent malicious cyber attacks.

Recently, federated learning (FL) has been extensively employed in distributed intrusion detection, particularly for securing large-scale and distributed IIoT-enabled systems (Li et al., 2020; Tahir et al., 2021; Idrissi et al., 2023; Zhang et al., 2023). This approach capitalizes on the edge intelligence paradigm, where each edge node processes IIoT-

generated data locally and utilizes deep learning methods to acquire intrusion detection models. The local processing crucially maintains data privacy, as sensitive information remains at the source site rather than transmits over the network. These local intrusion detection models are uploaded to the cloud server for global knowledge aggregation, the obtained global intrusion detection model is then broadcast to the clients for enhancing local intrusion detection. On the whole, FL develops a satisfactory solution to implement distributed intrusion detection with the privacy-preserving capabilities and the low communication costs, which has emerged as a powerful tool in the realm of IIoT cybersecurity.

However, the IIoT-enabled systems are actually deployed in a highly-complex and dynamic distributed environment (Wu et al., 2021), the application of FL-based intrusion detection in IIoT still confronts with two inevitable obstacles: First, with the dynamic evolution of cyber-attacks, FL-based intrusion detection model trained with historical traffic data is no longer capable of handling these new attacks encountered in current traffic data stream. More unfortunately, the limited resources of edge node render it infeasible to retrain and update the model from scratch (Gyamfi and Jurecut, 2022). Second, due to the

* Corresponding author.

E-mail address: zzhhaa156@gmail.com (L. Tang).

<https://doi.org/10.1016/j.engappai.2024.108826>

Received 21 January 2024; Received in revised form 26 May 2024; Accepted 7 June 2024

Available online 14 June 2024

0952-1976/© 2024 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

fact that the cyber-attacks present the characteristics of low-frequency and sudden-occurrence, it is unrealistic for the edge node to collect sufficient traffic data and determine the number of all the attack categories possibly encountered in the initial training stage. To tackle these obstacles, we propose the evolutionary distributed intrusion detection system based on federated continual representation learning, namely EvoFedIDS, as illustrated in Fig. 1. It facilitates local intrusion detection model continually learning the feature representations of new attacks from dynamic traffic data streams and identifying all the attacks that have encountered so far across all the clients. By leveraging class prototypes for intrusion detection, EvoFedIDS gets rid of prior knowledge of the number of all the attack categories that possibly encountered.

For the proposed EvoFedIDS, the key issue is to learn effective and discernible embedding representations from dynamic traffic data streams across clients. Firstly, we design the supervised contrastive loss on current traffic data and memory replay data, to progressively learn emerging attacks without catastrophic forgetting on previously encountered attacks. Concretely, we introduce the memory buffer and propose the prototype variance-based memory update strategy to assure the effectiveness of the memory replay data. Additionally, by incorporating the information of the global model and global model parameter importance into local training, we propose the global information-aware regularization loss to alleviate the negative impact of heterogeneity across clients. The main contributions of this paper are manifold:

- For the first time, we propose federated continual representation learning to build the evolutionary distributed intrusion detection systems in IIoT-enabled systems.
- We design the supervised contrastive loss on the current traffic data stream and the memory replay data, to learn effective representations of the new attacks and retain the discriminations on the previously encountered attacks.
- To alleviate the negative impact of heterogeneity, we propose the global information-aware regularization loss, via introducing the global model and global model parameter importance in training. Notably, we propose the gradient-based model parameter importance estimation method.
- We propose the prototype variance-based memory update strategy, ensuring the effective memory replay data with a limited memory size.

The rest of this paper is organized as follows. Section 2 provides an overview of related works. The threat model is discussed in Section 3. Section 4 introduces the proposed EvoFedIDS. Experimental results are presented in Section 5. Finally, Section 6 comes to conclusion remarks.

2. Related work

2.1. FL-based intrusion detection in IIoT

Network intrusion detection technology (Ling and Hao, 2022a,b) based on deep learning has gained widespread adoption in defending against cyber-attacks targeting IIoT-enabled systems. Song et al. (2020) proposed an intrusion detection system for Internet of Energy (IoE) based on a residual network with attention long short-term memory and validated its effectiveness on a real IoE dataset, called gas pipeline dataset. As for the blockchain-based energy network, the intrusion detection system based on recurrent neural network was proposed to detect cyber-attacks and fraudulent transactions (Ferrag and Maglaras, 2019). Results on CICIDS2017 dataset and Bot-IoT dataset verified its superior performance. Boppana and Bagade (2023) proposed unsupervised intrusion detection model based on generative adversarial networks and autoencoder for IoT applications. The aforementioned approaches are typically implemented in a central manner, where all the network traffic is gathered together and analyzed on a central server. This centralized approach raises concerns regarding sensitive

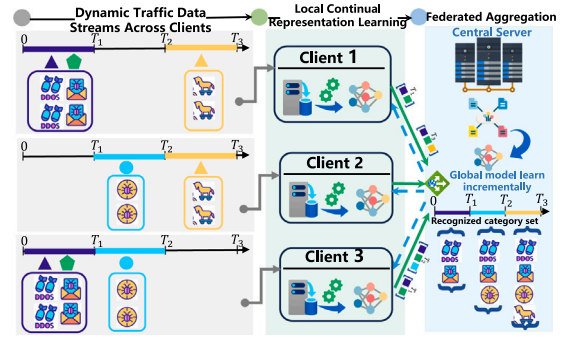


Fig. 1. Scenario illustrations of EvoFedIDS.

information leakage and privacy issues. In contrast, our work focuses on distributed intrusion detection, which aims to address this limitation of centralized approaches.

Recently, FL-based distributed intrusion detection has been proposed to protect the security of large-scale distributed IIoT/IoT. Li et al. (2020) proposed an intrusion detection system based on FL in IIoT, DeepFed. It exploited CNN combined with a gated recurrent unit to learn local representations, and utilized FL to leverage local data resources from multiple industry owners. Tahir et al. (2021) proposed a decentralized attack detection method based on FL, which can reliably identify attack on all nodes at the same time without affecting the data privacy of each client. The proposed Fed-ANIDS (Idrissi et al., 2023) leveraged the anomaly detection method and the FL framework (FedProx) to address the privacy concerns associated with centralized intrusion detection models. In Zhang et al. (2023), a federated continual learning method called FedSI was applied to distributed intrusion detection and achieved improved performances on CICIDS2017 dataset. But these works embrace the implicit assumption that the attack categories are fixed and unchanged over the training time, in fact, FL-based distributed intrusion detection is deployed in a dynamic IIoT environment. This assumption does not reflect the dynamic and evolving nature of cyber threats as encountered in real-world scenarios. To address this gap, we propose the evolutionary distributed intrusion detection system on dynamic traffic data streams.

2.2. FL-based intrusion detection with dynamic data

The current intrusion detection methods often combine continual learning to address dynamic traffic data. Prasath et al. (2022) investigated the potential of continual learning models for incrementally learning new data patterns while retaining previously learned information. Cerasuolo et al. (2023) proposed adaptive intrusion detection systems based on class incremental learning, employing Incremental Classifier and Representation Learning (iCaRL) to adapt to new attack categories in IIoT. Karn et al. (2021) expanded the application domain of Synaptic Intelligence (SI) from imaging to network cybersecurity, enabling incremental detection of network threats. In Xu et al. (2024), a self-paced class incremental learning method was introduced to maintain the discrimination of previously encountered malware categories as new categories emerge. Even though these methods have made substantial advancements in this field, it is crucial to acknowledge that they predominantly focus on centralized intrusion detection systems. Upon adapting them to FL-based intrusion detection systems, these methods encounter the significant challenges from the catastrophic forgetting and data heterogeneity that critically impede the model's performance.

Designing FL system under the dynamic environment is still an open issue with no semblance of a unified solution. Federated class-incremental learning (FCIL) was proposed to adapt for processing dynamic data streams (Dong et al., 2022), where each local client collects

the training data of new classes continuously and can join in the federated training at any time. It developed the global-local forgetting compensation model, to learn a global class-incremental model for alleviating catastrophic forgetting. Zhang et al. (2022a) proposed the decentralized composite knowledge incremental distillation framework to achieve decentralized class-incremental learning. Hendryx et al. (2021) proposed federated reconnaissance based on prototypical networks, namely Fedproto. It enabled the distributed clients to learn a growing set of new classes independently and communicate that knowledge efficiently. These works can provide the benchmarks in the dynamic federated learning scenario. In this paper, we compare the performances of our proposed EvoFedIDS with these of the state-of-the-art methods.

3. Threat model

The IIoT-based technologies have significantly boosted the efficiency of monitoring and operating critical infrastructures, but the deployment of IIoT technologies has also triggered cyber-security concerns. In this section, we discuss the following attacks against IIoT-enabled system (Zhang et al., 2021):

(1) Network-based attack: Adversaries utilize the TCP/IP Internet protocols, Modbus protocols, and other protocols to launch attacks against smart energy infrastructures, including DDos attack, SQL and XSS injection attack, etc.

(2) Control signal-based attacks: By bypassing data authentication and integration examinations, adversaries seek to operate the mission-critical physical devices in malicious manners. For instance, adversaries launch the malicious parameter command injection (MPCI) attack by altering the set points parameters in field devices, e.g., PID controller parameters.

(3) Measurement-based attacks: This type of attack is targeted to manipulate the sensor measurements data transmitted over communication channels or forge the remote terminal units in the field. Adversaries modify the device firmware, eavesdrop measurements for reconnaissance, and control sensors for reporting tampered measurements.

These attacks may cause the key information leakage and provoke major safety accidents within IIoT-enabled system, thereby it is essential to employ the powerful intrusion detection to defend these potential attacks.

4. The proposed EvoFedIDS framework

4.1. System architecture

In this paper, we build the evolutionary distributed intrusion detection system for protecting the network security of IIoT-enabled systems, called EvoFedIDS. To implement EvoFedIDS in realistic IIoT environment, we first design a multi-layer system architecture, integrating emerging technologies such as smart sensors, software defined network (SDN), and edge computing, as depicted in Fig. 2,

(1) IIoT layer: This physical layer is equipped with a series of IIoT devices and the advanced metering infrastructures for monitoring and collecting environmental information.

(2) Edge layer: Edge nodes act as the local intrusion detection clients. Generally, the smart security gateway is served as the edge computing device. It processes the traffic data and sensor data transmitted from the source sites and then trains its own local intrusion detection model.

(3) Cloud server: It is responsible for gathering the knowledge transmitted from the local clients to obtain the global intrusion detection model, and then the global model is sent back to each edge node to enhance local intrusion detection.

(4) SDN controller: It instructs network rules to OpenFlow-enabled gateways, which are utilized to redirect the network traffic to the intrusion detection engine for inspecting the existence of malicious behaviors.

4.2. Problem definition

The proposed EvoFedIDS consists of N local intrusion detection clients and a central server. Considering the dynamics of traffic data, we assume that each client i collects a sequence of traffic data stream $D_i = \{D_i^1, D_i^2, \dots, D_i^t\}$ as its private local training dataset, where t is the time stage. That is, the training traffic data are collected from different stages, not directly acquired at the initial stage. Client i can only access to $D_i^t = \{(\mathcal{X}_i^t, \mathcal{Y}_i^t) | \mathcal{Y}_i^t \in C_i^t\}$ at stage t , where $\mathcal{X}_i^t$ is the training data of the attack categories C_i^t newly collected at t . Besides, due to the dynamic evolution of attacks, we assume that the attack categories newly collected at t vary from the previously collected categories, that is, $C_i^t \not\subseteq \bigcup_{l=1}^{t-1} C_i^l$. During stage t , the clients that own the training data from the same new categories participant in the FL process, denoted as $\mathcal{A}^t \subseteq \{1, \dots, N\}$. To mitigate the catastrophic forgetting problem, we also introduce a memory buffer $\mathcal{M}_i^t$ with a limited size to reserve a small subset of training data of the previously encountered attacks.

Our goal is to learn the global intrusion detection model that can identify all the categories that have encountered so far across all the clients, $C' = \bigcup_{i=1}^N (\bigcup_{t=1}^T C_i^t)$. We define $f_\theta(x) : \mathcal{X} \rightarrow \mathcal{Y}, \mathcal{X} \in D_i^t \cup \mathcal{M}_i^t, \mathcal{Y} \in C'$, the intrusion detection model is parameterized by θ . The optimization objective for θ typically encompasses two main aspects: learning the newly encountered attack categories and maintaining the identification for the previously encountered categories.

4.3. Proposed federated continual representation learning

To achieve the goal of EvoFedIDS, we propose federated continual representation learning (FCRL). Specifically, each local client equips with an embedding network that continually learns distinguishable representations of new attacks from dynamic traffic data streams and communicates the learned knowledge with the central server. Following the knowledge aggregation by the server conduct, the obtained global intrusion detection model is expected to discriminate the attacks encountered across all the clients. Essentially, FCRL comprises two crucial components: local continual representation learning and federated aggregation.

4.3.1. Local continual representation learning with dynamic traffic

The key issue is to learn effective and discernible representations from dynamic traffic data streams across local clients. Driven by the urgent need, we design the supervised contrastive loss and the global information-aware regularization loss to jointly optimize the local embedding network parameterized by θ .

Supervised contrastive learning (Khosla et al., 2020) is a newly developed representation learning method for the embedding network. It explicitly encourages samples from the same class to cluster tightly in the embedding space while pushing those of different classes further apart. Herein, we apply the supervised contrastive loss on the current traffic data stream and the memory replay data, intending to enable the local embedding network to learn the new attacks and meanwhile preserve the performance on identifying the previously encountered attacks. To be specific, the current traffic data stream contains the newly encountered attack categories and the memory buffer stores a subset of traffic data from the previously encountered categories. The proposed supervised contrastive loss $\mathcal{L}_{SC}$ is calculated by,

$$\mathcal{L}_{SC} = - \sum_{x_j \in \tilde{D}_i^t} \frac{1}{|\{x_j^+\}|} \sum_{x_p \in \{x_j^+\}} \log \frac{\exp(d(\mathbf{F}(\theta_i^t; x_j), \mathbf{F}(\theta_i^t; x_p))/\tau)}{\sum_{x_a \in (\tilde{D}_i^t \setminus x_j)} \exp(d(\mathbf{F}(\theta_i^t; x_j), \mathbf{F}(\theta_i^t; x_a))/\tau)} \quad (1)$$

where $\tilde{D}_i^t = D_i^t \cup \mathcal{M}_i^t$, x_p denotes as the positive sample of x_j , $\{x_j^+\} = \{x_k | y_k = y_j, k \neq j\}$ is the set of all the positive samples of x_j . x_a belongs to the set of all samples in $\tilde{D}_i^t$ except for x_j . $d(z_i, z_j)$ represents the distance function, we adopt the cosine similarity $d(z_i, z_j) = \frac{z_i^T z_j}{\|z_i\| \|z_j\|} \cdot \tau$ is the temperature factor.

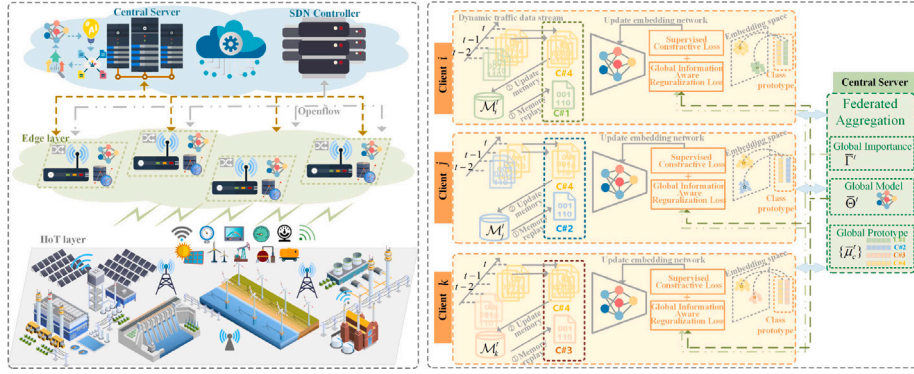


Fig. 2. The whole framework of EvoFedIDS.

Owing to the heterogeneity of IIoT devices, not each client can discover new categories and participate in the FL process during each stage. The participating clients own the training data from the same new categories at stage t , but their memory data may contain varying categories that have encountered in the previous stages, as shown in Fig. 2. This memory data heterogeneity will bring in non-independent identically distributed (Non-IID) training data, thereby exacerbating the catastrophic forgetting for the previously encountered categories and inducing the model performance degradation. In order to alleviate these negative influences of data heterogeneity, we propose the global information-aware regularization (GIAR) loss $\mathcal{L}_{GIAR}$, as in (2). As the global model acquired in the previous stage is capable of identifying all the previously encountered categories and covering their global information, it can be exploited to guide the local training and mitigate the catastrophic forgetting. By incorporating the information of the global model and global model parameter importance in local training, the proposed GIAR loss forces the local model parameter updates to be close to the global model on the important model parameters. Consequently, it pushes all the local models toward the global optima and thereby enhance the model performance on heterogeneous data.

$$\mathcal{L}_{GIAR} = \sum_k \bar{F}_k^t \|\Theta_{i,k}^t - \bar{\Theta}_k^t\|_2^2 \quad (2)$$

where $\bar{F}_k^t$ is the k th global model parameter importance, $\Theta_{i,k}^t$ denotes as the k th local model parameter in client i , and $\bar{\Theta}_k^t$ is the k th global model parameter.

The gradient-based model parameter importance estimation method is proposed to calculate the value of $\bar{F}_k^t$. The importance of model parameters can be quantified by the change in loss caused by an infinitesimal change in the model parameters. The greater the change in loss, the more important the model parameters are. Typically, the change in loss is estimated by the model gradient and the model parameter update (Zenke et al., 2017). The total change in loss caused by each parameter across the entire parameter space can be calculated by summing the changes in loss value across all local training iterations. For the local model, the importance of the model parameters can be estimated as follows:

$$\Gamma_{i,k}^t = \frac{\left| \int_t g_{i,k}^t \Delta \Theta_{i,k}^t \right|}{\left| \int_t \Delta \Theta_{i,k}^t \right|^2 + \xi} \quad (3)$$

where $\Gamma_{i,k}^t$ is the importance factor of the k th local model parameter in client i , $\Delta \Theta_{i,k}^t$ represents the k th local model parameter update, $g_{i,k}^t$ is the gradient of k th local model parameter, $g_{i,k}^t \Delta \Theta_{i,k}^t$ is the loss change caused by each parameter change, $\int_t$ means the cumulative changes during t . $\left| \int_t \Delta \Theta_{i,k}^t \right|^2$ is to ensure that the GIAR loss term carries the same units as the supervised contrastive loss term and ξ is the damping factor that avoids the denominator to be zero.

Due to the fact that the important model parameters of the local clients may be inconsistent, the importance factors of all the parameters

of each local model $\Gamma_i^t = \{\Gamma_{i,1}^t, \dots, \Gamma_{i,k}^t, \dots, \Gamma_{i,|\Theta|}^t\}$ are uploaded to the central server, and then the central server aggregates all the local models' information to obtain the global model parameter importance,

$$\bar{F}_k^t = \sum_{i \in \mathcal{A}^t} \frac{n_i^t}{n^t} \Gamma_{i,k}^t \quad (4)$$

where $\mathcal{A}^t$ is the set of the local clients participating in stage t , n_i^t is the number of instances in client i , and n^t is the number of instances over all clients.

Overall, the optimization objective of client i that involves the local continual representation learning is written as follows,

$$\mathcal{L}_i = \mathcal{L}_{SC} + \lambda \mathcal{L}_{GIAR} \quad (5)$$

where $\mathcal{L}_{SC}$ is the supervised contrastive loss, $\mathcal{L}_{GIAR}$ is the global information-aware regularization loss, λ is the scalar to balance the two loss term.

Then, the local client updates the local model parameters by minimizing the local training loss, which is expressed as:

$$\Theta_i^{t+1} = \Theta_i^t - \eta \nabla \mathcal{L}_i(\Theta_i^t; \tilde{\mathcal{D}}_i^t) \quad (6)$$

where η is the learning rate. During each communication round r , the local client conducts multiple local training epochs E to update the local model parameters.

Upon updating the local embedding model, it is observed that the learned embeddings of the samples within the same class are clustered together and well-separated from those with different classes. Inspired by this observation, we leverage the mean of the embeddings from the same class as the class representative to assist intrusion detection, called the class prototype,

$$\mu_{i,c}^t = \frac{1}{n_{i,c}^t} \sum_{x_j \in \tilde{\mathcal{D}}_i^t} \mathbf{F}(\Theta_i^t; x_j) \cdot \mathbb{1}\{y_j = c\} \quad (7)$$

where $\mu_{i,c}^t$ is the class prototype of class c in local client i during stage t , $n_{i,c}^t$ is the number of instances belonging to class c in $\tilde{\mathcal{D}}_i^t$, and $\mathbb{1}\{y_j = c\}$ is the indicator for $\{y_j = c\}$. All the class prototypes in local client $\{\mu_{i,c}^t | c \in \mathcal{C}_i^t\}$ are transmitted to the server for knowledge sharing and merging.

Lastly, once the local training is completed at stage t , the memory buffer needs to be updated with the current traffic data stream. Given the limited size of the memory buffer, we propose the prototype variance-based memory update strategy. To ensure the memory replay effectively, it is crucial to allocate the sufficient memory space for those classes that are not easily distinguishable. If the variance of the embeddings within a class is large, more samples of this class are required for training an efficient intrusion detection model. Conventionally, the variance of the class prototype can serve as a guideline for the memory allocation, thereby we adopt the standard deviation as a measure to

calculate the variance of the class prototype $\sigma_{i,c}^t$,

$$\sigma_{i,c}^t = \sqrt{\frac{\sum_{j=1}^{n_{i,c}^t} (\mathbf{F}(\Theta_i^t; x_j) - \mu_{i,c}^t)^2}{n_{i,c}^t}} \quad (8)$$

where x_j is the instance belongs to class c in local client i . Based on this, we allocate a sufficient portion of memory to the class with a large variance. By introducing the memory partition factor $\beta_{i,c}^t$ for class c , the corresponding memory partition size $m_{i,c}^t$ is obtained by (10),

$$\beta_{i,c}^t = \frac{\sigma_{i,c}^t}{\sum_{c \in \bigcup_{i=1}^t C_i^t} \sigma_{i,c}^t} \quad (9)$$

$$m_{i,c}^t = \beta_{i,c}^t |\mathcal{M}_i^t| \quad (10)$$

where $\beta_{i,c}^t$ is the memory partition factor for class c , $m_{i,c}^t$ is the memory partition size for class c , $|\mathcal{M}_i^t|$ is the size of the whole memory buffer. According to the memory partition size, we adopt the random sampling to select samples into the memory buffer for the next round. The details of the proposed memory update strategy are illustrated in Algorithm 1.

4.3.2. Federated aggregation for global intrusion detection

As the typical FL suggested, the central server conduct the global aggregation for all the updated local model parameters to obtain the global embedding network $\bar{\Theta}^{t+1}$,

$$\bar{\Theta}^{t+1} = \sum_{i \in \mathcal{A}^t} \frac{n_i^t}{n^t} \Theta_i^{t+1} \quad (11)$$

In addition to global model aggregation, the central server also conducts the global prototype aggregation that effectively integrate the data representations from diverse classes. The local client uploads the calculated class prototype to the central server, and the central server aggregates all the class prototypes to acquire the global prototype $\bar{\mu}_c^t$,

$$\bar{\mu}_c^t = \sum_{i \in \mathcal{A}^t} \frac{n_{i,c}^t}{n_c^t} \mu_{i,c}^t \quad (12)$$

where $n_{i,c}^t$ and n_c^t are the number of instances belonging to class c in client i and over all clients, respectively. Subsequently, the central server delivers the global intrusion detection model and the global class prototype to each local client, thus achieving the knowledge sharing across clients.

During the intrusion detection phase (model inference), we employ the prototype distance-based intrusion detection method, which is scalable to a growing set of attack categories. Concretely, each local client compares the embeddings of the detected traffic sample x with all the global class prototypes and categorizes it as the class with the shortest distance:

$$y^* = \underset{c \in \bigcup_{i=1}^N (\bigcup_{i=1}^t C_i^t)}{\operatorname{argmin}} \|\mathbf{F}(\bar{\Theta}^t; x) - \bar{\mu}_c\| \quad (13)$$

where $\bigcup_{i=1}^N (\bigcup_{i=1}^t C_i^t)$ denotes as all the categories that have encountered so far all the local clients (abbreviated to C^t).

4.4. The workflow of EvoFedIDS

The core idea of EvoFedIDS is to implement the local continual representation learning at the local client side and the global aggregation at the central server side, the complete workflow of the proposed EvoFedIDS is shown in Algorithm 1. It can be divided into three main parts:

- (1) **Determining system input:** At the start of each round r , each client reports the current training data category and the presence of any new attack category to the server. By gathering the information, the server determines the participating local client set $\mathcal{A}^t$ in the current round.

Algorithm 1: The workflow of EvoFedIDS

Input: Local clients set $\mathcal{A}^t$ participating in current stage t , local training datasets $\{D_i^t | i \in \mathcal{A}^t\}$ and local memory buffers $\{\mathcal{M}_i^t | i \in \mathcal{A}^t\}$

Output: Global model and global class prototypes

```

1 for each round  $r = 1$  to  $R$  do
2   // Local training at client side ;
3   for each client  $i$  in  $\mathcal{A}^t$  do
4     Calculate local training loss  $\mathcal{L}_i$  based on (1)–(5);
5      $\Theta_i^{t+1} \leftarrow \Theta_i^t - \eta \nabla \mathcal{L}_i(\Theta_i^t; \bar{D}_i^t)$ ;
6     Calculate  $\{\mu_{i,c}^t | c \in C_i^t\}$  and  $\Gamma_i^t$ ;
7     Send  $\{\mu_{i,c}^t | c \in C_i^t\}$ ,  $\Theta_i^{t+1}$ , and  $\Gamma_i^t$  to the central server;
8      $\mathcal{M}_i^{t+1} \leftarrow \text{MemoryUpdate}(D_i^t, \mathcal{M}_i^t)$ ;
9   // Global aggregation at server side;
10  Obtain global model  $\bar{\Theta}^{t+1}$ ,  $\{\bar{\mu}_c^t | c \in \bigcup C_i^t\}$ , and  $\bar{\Gamma}^t$ ;
11  Broadcast them to the local clients for next round;
12 Function MemoryUpdate( $D_i^t, \mathcal{M}_i^t$ ):
13   Calculate  $\sigma_{i,c}^t$  for each class prototype;
14   for  $c$  in  $C_i^t$  do
15     Calculate the memory size  $m_{i,c}^t$  based on (9)–(10);
16      $n_{i,c}^t \leftarrow$  Number of class  $c$  samples in  $D_i^t \cup \mathcal{M}_i^t$ ;
17     for  $j = 1$  to  $n_{i,c}^t$  do
18       Sample  $u \sim \text{Uniform}(0,1)$ ;
19       if  $u \leq m_{i,c}^t / n_{i,c}^t$  then
20         Select  $(x_j, y_j)$  into  $\mathcal{M}_i^{t+1}$ ;
21       else
22         Ignore it;
23   return  $\mathcal{M}_i^{t+1}$ ;

```

- (2) **Local continual representation learning by clients:** The local clients utilize the delivered global intrusion detection model as the initial model parameters for the current round. By training the designed local loss $\mathcal{L}_i$, the local client obtains the updated local embedding network model Θ_i^{t+1} , the class prototypes $\{\mu_{i,c}^t | c \in C_i^t\}$, and the importance factors of the local model Γ_i^t , and then upload them to the server. After the local training, the memory buffer $\mathcal{M}_i^t$ is updated according to the proposed prototype variance-based memory update strategy.
- (3) **Global knowledge aggregation by the central server:** By performing the average aggregation, the server acquires the global local embedding network model $\bar{\Theta}^{t+1}$, the global class prototypes $\{\bar{\mu}_c^t | c \in \bigcup C_i^t\}$, and the importance factors of the global model $\bar{\Gamma}^t$, and then broadcasts these global parameters to all clients for the subsequent rounds.

In summary, this framework allows for collaborative learning while taking into account the dynamic traffic data streams across local clients.

5. Experiments and analysis

5.1. Dataset and experimental settings

5.1.1. Datasets

The gas pipeline system is a typical application scenario for IIoT-enabled smart energy system. In this paper, to evaluate the performance of EvoFedIDS, we employ a real gas pipeline dataset (Morris and Gao, 2014). This dataset is collected from the Supervisory Control and Data Acquisition (SCADA) system of a gas pipeline test platform, providing a realistic representation of the operational data in such environments. Comprising 27 features, it encompasses both network traffic features

Table 1

Data settings of dynamic traffic on gas pipeline dataset and CICIDS2018 dataset.

Gas pipeline dataset			CICIDS2018 dataset		
	Encountered categories	$ C' $		Encountered categories	$ C' $
Stage1	Normal,ComplexMalicious ResponseInjection(CMRD),Malicious	3	Stage1	Normal,Bot, DosHulk	3
	StateCommandInjection(MSCI)		Stage2	FTPBruteForce	4
Stage2	MaliciousParameter	4	Stage3	SSHBruteforce	5
	CommandInjection(MPCI)		Stage4	DDoSHOIC	6
Stage3	MaliciousFunction	5	Stage5	DoSGoldenEye	7
	CodeInjection(MFCI)		Stage6	DoSSlowloris	8
Stage4	Dos	6	Stage7	DDoSLOIC	9
Stage5	Reconnaissance	7	Stage8	BruteForceXSS	10

 $|C'|$ is the number of categories that have encountered so far.

and payload content features. The network traffic features are leveraged to reflect the communication status of SCADA system, and the payload features include current measured values/initial values of gas pressure, system control modes, etc. Additionally, to further validate the effectiveness of EvoFedIDS, we conduct extensive experiments on CICIDS2018 dataset. This dataset is collected from a large-scale test platform and encompasses a wide range of novel network attacks, which provides a comprehensive representation of realistic network environments.

In our experiments, we adhere to the data preprocessing procedures outlined in Zhang et al. (2022b). Each dataset is divided into two parts: 70% for training and the remaining 30% for testing. The experimental data settings for both gas pipeline dataset and CICIDS2018 dataset are summarized in Table 1.

5.1.2. Experimental settings

The proposed EvoFedIDS is designed with a distributed system architecture, consisting of one central server and several local clients. At each stage, considering that not all clients may acquire training samples of newly encountered attacks, we randomly select 5 clients to participate in the FL process. The local intrusion detection model is an embedding network, architecturally based on CNN, comprising two convolutional layers accompanied by max-pooling layers. The output feature embedding dimension is fixed to 8. In addition, we incorporate a memory replay mechanism that allows to retain knowledge from previous stages. The memory size is set to 500 for gas pipeline dataset and 2000 for CICIDS2018 dataset. We implement the proposed EvoFedIDS using PyTorch framework and Python. The source codes of EvoFedIDS implementation are released online (<https://github.com/zhangzhao156/EvoFedIDS>).

5.1.3. Benchmarks

To verify the superiority of the proposed method, we compare the performances of EvoFedIDS with state-of-art FL-based intrusion detection method and dynamic federated learning methods. The implementations of these methods are based on the descriptions of their papers. Note that these dynamic federated learning methods are the first time to be applied to FL-based intrusion detection. The benchmarks chosen for comparison are outlined below:

- FedProx (Idrissi et al., 2023): In Idrissi et al. (2023), the proposed Fed-ANIDS utilized FedProx to implement FL for network intrusion detection systems. By introducing a proximal term in the local training objective, FedProx significantly bolsters the generalization performance in federated environments.
- FedSI (Zhang et al., 2023): FedSI was applied to distributed intrusion detection system (Zhang et al., 2023), combining FL with continual learning. Through the meticulously designed loss function, FedSI improved the performance on non-independent and identically distributed (Non-IID) traffic data.

- FedSpace (Shenaj et al., 2023): FedSpace was an asynchronous federated continual learning method that enables clients to learn multiple tasks in varying orders (Shenaj et al., 2023). By jointly training the local model with supervised cross-entropy loss, prototype-based loss, and contrastive representation loss, FedSpace facilitates the adaptation to evolving data distributions over time.
- Fedicarl (Dong et al., 2022): Fedicarl belongs to the dynamic federated learning method that integrates the typical class incremental learning method iCaRL into a FL framework, exemplifying adaptability in dynamic environments.
- FCIL (Dong et al., 2022): FCIL also falls under the dynamic federated learning umbrella. It designs the global-local forgetting compensation loss in the local training, tailored to suit dynamic environments.
- Fedproto (Hendryx et al., 2021): Fedproto belongs to the dynamic federated learning method. It leverages the prototypical networks, empowering distributed clients to independently learn a growing set of new classes.

5.1.4. Evaluation indicators

To evaluate the performance of EvoFedIDS on dynamic traffic data streams, we employ the detection accuracy of the global intrusion detection model obtained at each stage as a key metric. Considering of the evolving attack threats, each stage corresponding to a unique test dataset that contains a varying number of attack categories, as reported in Table 1. Herein, we define the average accuracy across all stages as a holistic performance evaluation metric. Following previous works (Srivastava et al., 2023), we also consider the metrics of the backward transfer (BWT) score and the forward transfer (FWT) score. BWT Score measures the impact that learning new tasks has on the performance of previously learned tasks, while FWT Score assesses how prior learning influences the performance on future tasks before they are learned. The larger the values of BWT and FWT, the better. Overall, we utilize these metrics to assess the model performance. Their formulas are as follows:

$$\mathcal{R}_t = \frac{1}{|D_t|} \sum_i \mathbb{1}(\hat{y}_i, y_i), \bar{\mathcal{R}} = \frac{1}{T} \sum_{t=1}^T \mathcal{R}_t \quad (14)$$

$$BWT = \frac{1}{T-1} \sum_{t=1}^{T-1} \mathcal{R}_{T,t} - \mathcal{R}_{t,t} \quad (15)$$

$$FWT = \frac{1}{T-1} \sum_{t=2}^T \mathcal{R}_{t-1,t} - \bar{b}_i \quad (16)$$

where $\mathcal{R}_t$ denotes as the stage-specific detection accuracy, $\bar{\mathcal{R}}$ is the average accuracy across all stages, T denotes as the stage number, $\mathcal{R}_{i,j}$ is the test accuracy of the model on stage j after observing the last sample from stage i .

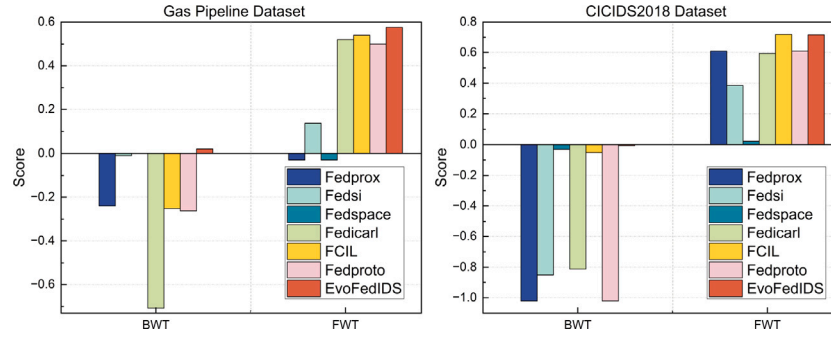


Fig. 3. BWT scores and FWT scores of all methods.

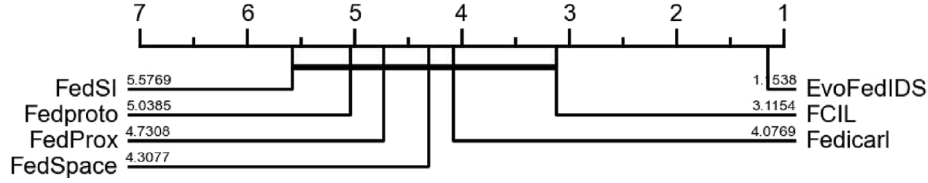


Fig. 4. Wilcoxon-Holm post-hoc test on datasets for all baseline methods.

Table 2

Performances of EvoFedIDS on gas pipeline dataset.

	Stage1	Stage2	Stage3	Stage4	Stage5	$\bar{R}$
Fedprox	98.16	89.24	72.31	70.79	65.66	79.23
FedsI	98.16	89.07	66.36	80.93	85.11	83.93
FedSpace	98.13	89.21	88.61	86.76	80.48	88.64
Fedicarl	98.18	9.09	96.96	50.96	75.65	66.17
FCIL	98.15	26.83	97.68	85.27	82.48	78.08
Fedproto	98.15	92.14	92.07	68.8	70.67	84.37
EvoFedIDS	98.17	97.99	97.12	97.61	97.63	97.67

5.2. Effectiveness evaluation

5.2.1. Performance evaluation of EvoFedIDS

We compare the performance of EvoFedIDS with the state-of-art methods as aforementioned across various stages. According to Table 2, EvoFedIDS consistently achieves high accuracy rates exceeding 97% on gas pipeline dataset. This performance demonstrates that EvoFedIDS effectively enables the global intrusion detection model to continually learn new attack categories from dynamic traffic data. Additionally, EvoFedIDS shows a substantial improvement in average accuracy, ranging from 13.3% to 31.5% over these state-of-art methods. Similarly, on CICIDS2018 dataset, as detailed in Table 3, EvoFedIDS maintains impressive accuracy levels between 96% and 99% throughout all stages. Notably, it surpasses the compared methods by more than 14.6% in average accuracy, despite CICIDS2018 dataset consisting more novel attack categories. Furthermore, Fig. 3 illustrates the BWT and FWT scores for various methods. EvoFedIDS stands out by achieving the highest gains in both BWT and FWT, indicating significantly reduced catastrophic forgetting of previously encountered attack categories and enhanced capability in adapting to new categories. Specifically, for the gas pipeline dataset, EvoFedIDS not only attains a positive BWT score but also substantially outperforms other methods. Overall, these results affirm the superiority of EvoFedIDS in adapting to and learning from dynamic traffic data, underscoring its effectiveness as a robust solution for the evolutionary distributed intrusion detection systems.

To ascertain the statistical significance of the compared methods, we employ the Friedman rank-sum nonparametric statistical test with a 95% confidence level. The null hypothesis asserts that there is no

statistical difference exist among the compared methods. Based on the average rank of all results as reported in Tables 2 and 3, the p -value obtained from the first test is $2.16e-06$. This value strongly suggests rejection of the null hypothesis, indicating that our method demonstrates statistically significant differences. To more precisely quantify the true differences between these methods, we utilize the Wilcoxon-Holm post-hoc test and illustrate the results in a critical difference diagram based on average ranks. Fig. 4 clearly displays the comparative performance of each method, emphasizing the statistically significant differences among them. Notably, most of the compared methods showed less optimal performance on both datasets. This refined analysis further reinforces the effectiveness of our approach in demonstrating meaningful advancements over existing methods.

Furthermore, we analyze the detection accuracy of each category at different stages across two datasets, as shown in Figs. 5 and 6. The results highlight several key observations. First, it is obvious that EvoFedIDS enables to gradually learn to identify more new attack categories and maintain high detection accuracies for all the previously encountered attack categories. Second, the detection performances of FedProx, FedSI, and FedSpace are comparatively lower, particularly when recognizing new attack categories. This issue primarily arises from their reliance on regularization terms from previous models, which stabilizes recognition of past data but hampers adaptation to newly emerging threats. Third, Fedicarl, FCIL, and Fedproto show declining accuracy for previously encountered categories, bearing the catastrophic forgetting problem to a certain extent. Notably, Fedicarl only recognizes new categories in Stage 2, forgetting previous ones and achieving only a 9% accuracy, as reported in Table 2. Fedproto performs well at the early stage but struggles as more categories appear because it does not update feature representations with new data. It emphasizes the necessity of continuously incorporating new category data into local model updates to maintain effective and adaptive detection capabilities.

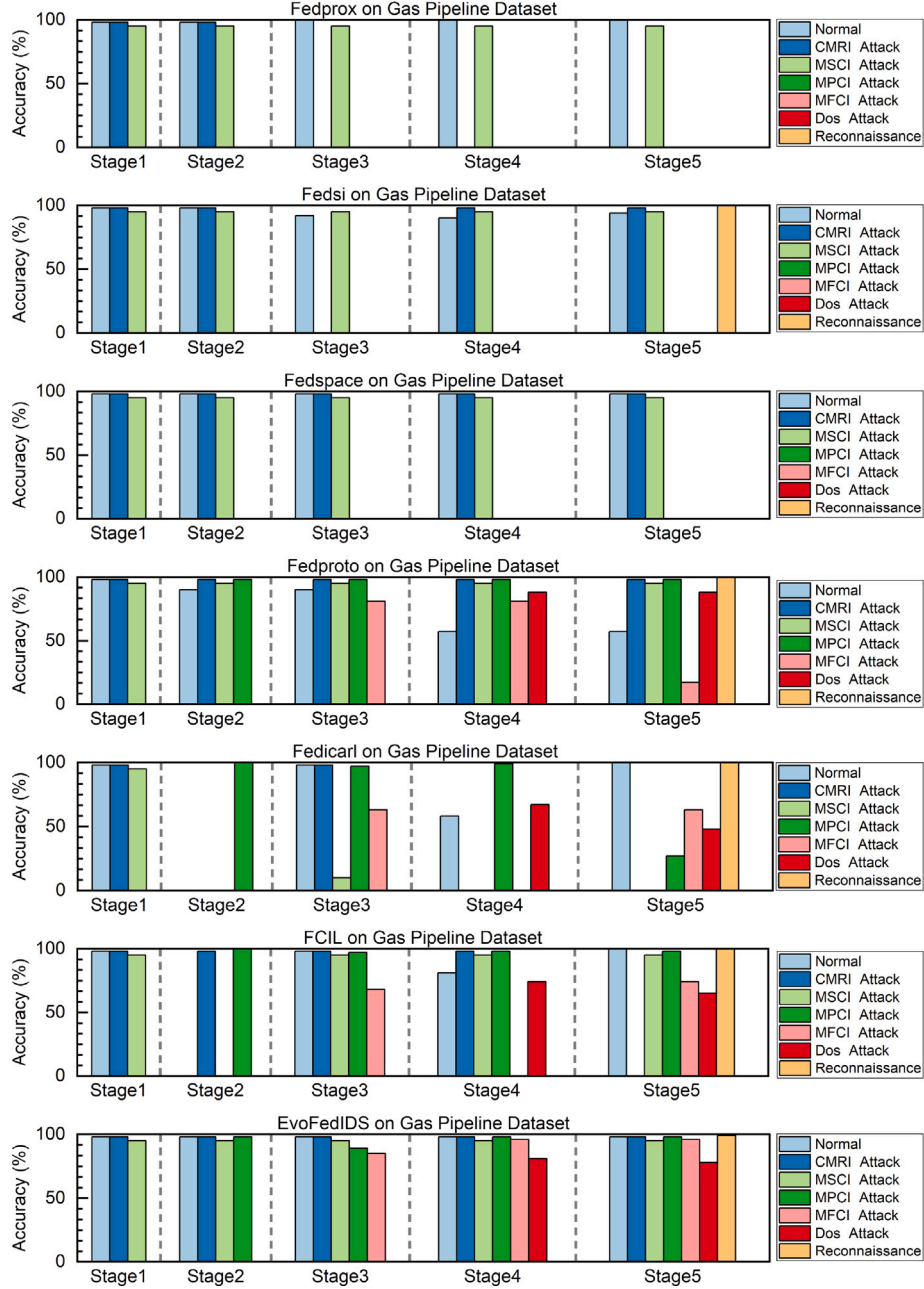
5.2.2. Robustness evaluation of EvoFedIDS

In a FL system, the central server cannot directly access local datasets, making it susceptible to poisoning attacks by malicious clients who modify local datasets (Zhang et al., 2022b). To assess the robustness of EvoFedIDS, we take into consideration two prevalent data poisoning attacks: the label flipping attack and the clean label attack.

Table 3

Performances of EvoFedIDS on CICIDS2018 dataset.

	Stage1	Stage2	Stage3	Stage4	Stage5	Stage6	Stage7	Stage8	$\bar{R}$
Fedprox	98.9	89.38	89.50	54.24	43.45	19.73	16.64	11.11	52.87
Fedsi	98.92	80.34	10.00	23.40	22.02	1.28	1.28	1.28	29.82
Fedspace	98.36	88.06	79.19	60.10	47.60	46.52	46.37	42.9	63.64
Fedcarl	96.13	88.71	84.15	67.37	63.09	65.71	73.18	36.87	71.90
FCIL	98.72	95.13	56.4	70.07	89.33	81.59	81.85	96.48	83.70
Fedproto	94.32	94.07	66.09	50.62	39.61	35.07	33.01	32.42	55.65
EvoFedIDS	99.59	99.69	98.9	96.5	96.48	98.09	98.35	98.69	98.29

**Fig. 5.** The attack detection accuracies of the proposed EvoFedIDS and the compared methods for each stage on Gas Pipeline Dataset.

Detailedly, the label flipping attack entails randomly changing the labels of the training traffic samples to other labels while keeping the traffic features unchanged. The clean label attack involves adversarially modifying the training traffic features while keeping the labels unchanged. Besides, poisoning attack is launched only in one round in our experiments.

We examine the impact of the poisoned sample number on the performance of EvoFedIDS under the label flipping attack and the clean label attack. Table 4 depicts the overall accuracies of EvoFedIDS with varying percentages of poisoned samples (P_a) in one client. For Gas pipeline dataset, the accuracy of EvoFedIDS declines significantly when more than 70% of the poisoned samples are injected under the clean

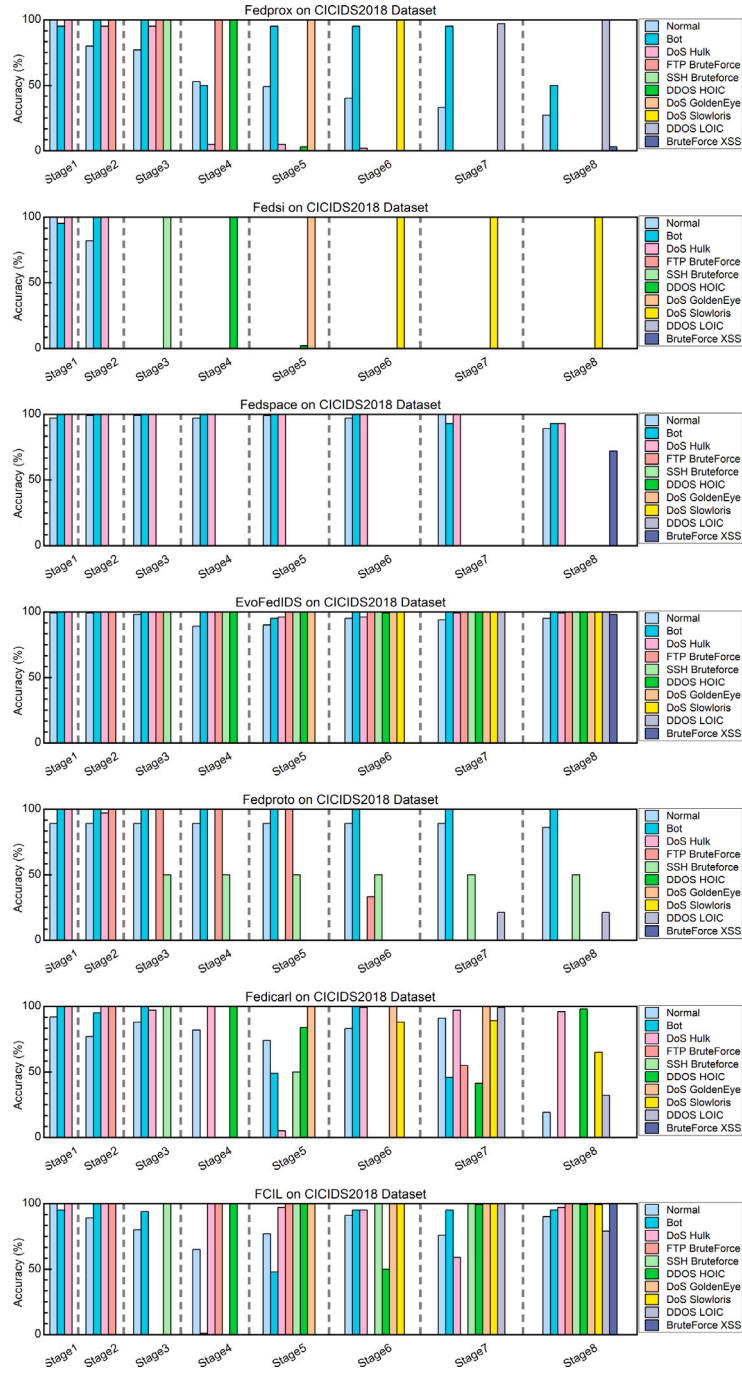


Fig. 6. The attack detection accuracies of the proposed EvoFedIDS and the compared methods for each stage on CICIDS2018 Dataset.

Table 4
Performances of EvoFedIDS under the label flipping attack and the clean label attack.

Setting	Gas pipeline dataset		CICIDS2018 dataset	
	Clean label attack	Label flipping attack	Clean label attack	Label flipping attack
$N_a=1$	$P_a=10\%$	97.51 ± 0.01	97.61 ± 0.01	97.77 ± 0.22
	$P_a=30\%$	97.51 ± 0.02	97.69 ± 0.06	98.06 ± 0.17
	$P_a=50\%$	97.56 ± 0.03	92.77 ± 5.68	97.56 ± 0.04
	$P_a=70\%$	97.47 ± 0.01	73.31 ± 8.79	97.81 ± 0.20
	$P_a=100\%$	89.57 ± 0.01	73.03 ± 3.52	96.86 ± 0.25
$P_a=30\%$	$N_a=1$	97.51 ± 0.02	97.69 ± 0.06	98.06 ± 0.17
	$N_a=2$	97.41 ± 0.04	92.21 ± 7.74	97.45 ± 0.96
	$N_a=3$	97.69 ± 0.03	94.09 ± 6.63	97.86 ± 0.06
	$N_a=4$	97.52 ± 0.04	93.21 ± 4.93	97.37 ± 0.62
	$N_a=5$	83.83 ± 11.73	92.08 ± 6.98	97.86 ± 0.07

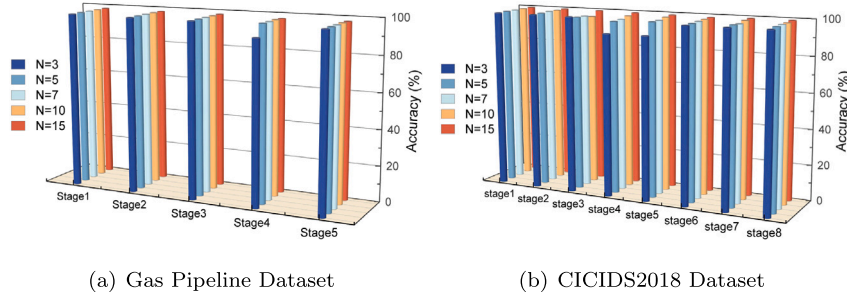


Fig. 7. Performances of the proposed EvoFedIDS with varying numbers of participating clients on Gas Pipeline Dataset and CICIDS2018 Dataset.

Table 5
Comparisons of EvoFedIDS ($N=1$) and EvoFedIDS ($N=5$) on two datasets.

	Gas pipeline dataset			CICIDS2018 dataset		
	Acc(%)	Training time (s)	Communication cost (KB)	Acc(%)	Training time (s)	Communication cost (KB)
$N=1$ (non-FL)	98.16	2.86	—	99.58	28.66	—
$N=5$ (FL)	98.15	0.82	66.76	99.3	9.24	290.76

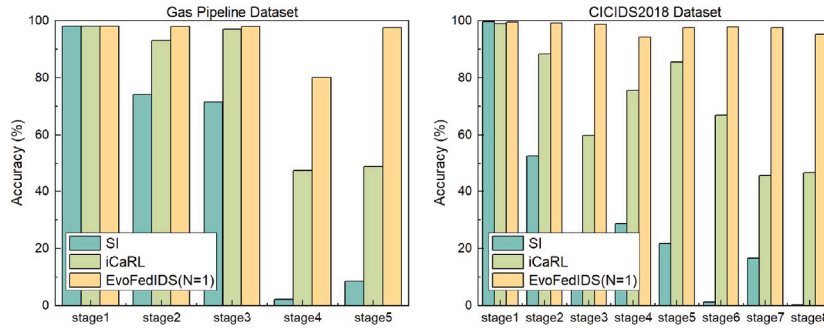


Fig. 8. Performances of the proposed EvoFedIDS ($N=1$) and the compared methods for each stage on Gas Pipeline Dataset and CICIDS2018 Dataset.

label attack and more than 50% under the label flipping attack. When only a few poisoned samples are injected into the local training dataset, their adverse effects are typically neutralized by the remaining clean models, thus minimally impacting EvoFedIDS's performance. Besides, Table 4 also lists the overall accuracies of EvoFedIDS with varying poisoned client number (N_a) and the fixed poisoned sample rate. The accuracy of EvoFedIDS does not significantly decrease with fewer than 5 poisoned clients. These results also validate the robustness of EvoFedIDS against the data poisoning attacks to some extent.

5.2.3. Scalability analysis of EvoFedIDS

To explore the scalability of EvoFedIDS, we conduct a comprehensive analysis focusing on its adaptability in two distinct scenarios: the large-scale IIoT scenarios (involving a vast number of clients) and the non-FL scenarios (involving a single client). Initially, we assess the deployment feasibility of EvoFedIDS in large-scale IIoT settings characterized by a vast number of clients. Fig. 7 illustrates the performance of EvoFedIDS at each stage with varying numbers of participating clients. The experimental results demonstrate that the performance of EvoFedIDS remains largely unaffected by the increase in client numbers. This observation highlights the importance of expanding client participation in bolstering the distributed system's stability and overall effectiveness in large-scale IIoT settings.

Subsequently, we adapt EvoFedIDS for a non-FL scenario with a single client, specifically setting $N=1$. Table 5 delineates the detection performance, training time, and communication costs per epoch for EvoFedIDS during Stage 1 with both $N=1$ and $N=5$ settings. Results indicate that the detection accuracy of EvoFedIDS in FL scenarios is comparable to that in non-FL scenarios, albeit with reduced training

time and additional communication overheads. This time efficiency in FL scenarios stems from the parallel training of local clients, and the ongoing communication overheads is due to the central server synchronizing model parameters for collaborative learning during training. In contrast, non-FL scenarios just transmitting all training data to a central server for centralized training, eliminating the need for communication during training. Furthermore, we also compare EvoFedIDS ($N=1$) with the exiting intrusion detection methods based continual learning, including iCaRL (Cerasuolo et al., 2023) and SI (Karn et al., 2021). As depicted in Fig. 8, our method consistently outperforms these comparative methods at various stages. Fig. 9 provides a classification confusion matrix to showcase the specific accuracy for each attack category at the final stage. It can be observed that EvoFedIDS ($N=1$) accurately detects most samples across all categories on both datasets. Although the detection accuracy for most categories exceeds 90% on Gas pipeline dataset, the accuracy for Dos attacks is lower at 70.81%, with many Dos samples misclassified as Normal type due to their similar patterns.

5.2.4. Key issues of EvoFedIDS

To further analyze the proposed EvoFedIDS, we first assess EvoFedIDS via visualizing the output embedding representations at each stage, and then we investigate the impacts of memory size, the proposed GIAR loss, task execution sequence, local training epochs (E), and communication round (R).

Representations visualizations: Figs. 10 and 11 demonstrate the visualizations of the output embedding representation based on T-SNE on gas pipeline dataset and CICIDS2018 dataset, respectively. It is apparent that the representations of the samples from the same

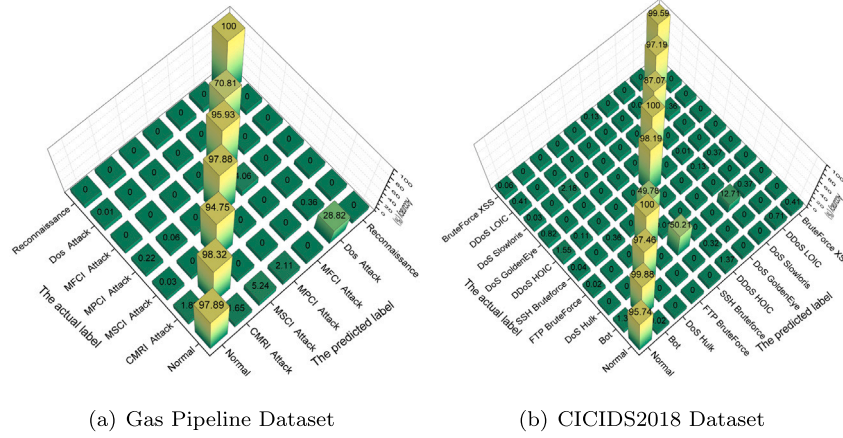


Fig. 9. The confusion matrix of the proposed EvoFedIDS (N=1) at the final stage on Gas Pipeline Dataset and CICIDS2018 Dataset.

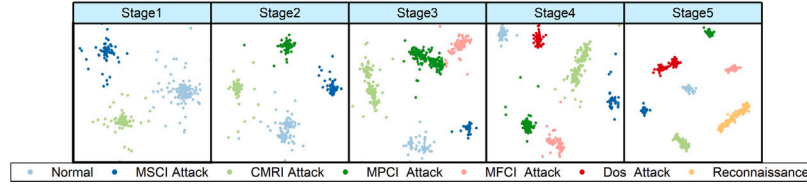


Fig. 10. The visualizations of the embedding representations at each stage on Gas Pipeline Dataset.

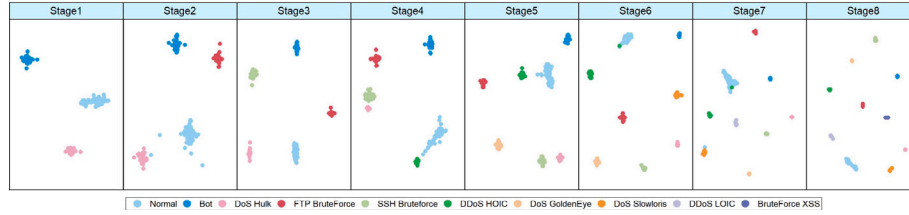


Fig. 11. The visualizations of the embedding representations at each stage on CICIDS2018 Dataset.

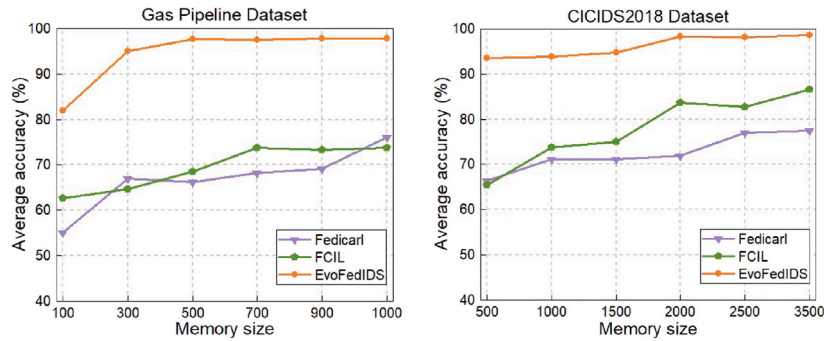


Fig. 12. Qualitative analysis of memory size on two datasets.

class cluster together and the representations from different classes are distributed far away, indicating that the proposed EvoFedIDS enables the local model to learn distinguishable and separable representations for the encountered categories progressively. Additionally, when new attacks are learned, the representation of the previously encountered attacks also evolves, herein it necessitates the recalculation and updating the class prototypes of the previously encountered attacks at each new stage.

Qualitative analysis of memory size: From Fig. 12 that the larger the memory buffer, the better the performance of the global intrusion

detection model when learning new attacks in a stream manner. Unlike FCIL and Fedicarl, the average accuracy of EvoFedIDS tends to be saturated when the memory size is greater than 500 on gas pipeline dataset and 2000 on CICIDS2018 dataset. This shows that EvoFedIDS can perform well with a smaller memory buffer size. In addition, this is why we chose 500 and 2000 as the memory buffer size on gas pipeline dataset and CICIDS2018 dataset, respectively.

Effect of the proposed GIAR loss: According to Fig. 13, the accuracies of EvoFedIDS without using the proposed GIAR loss decline as more attacks are encountered, especially on gas pipeline dataset,

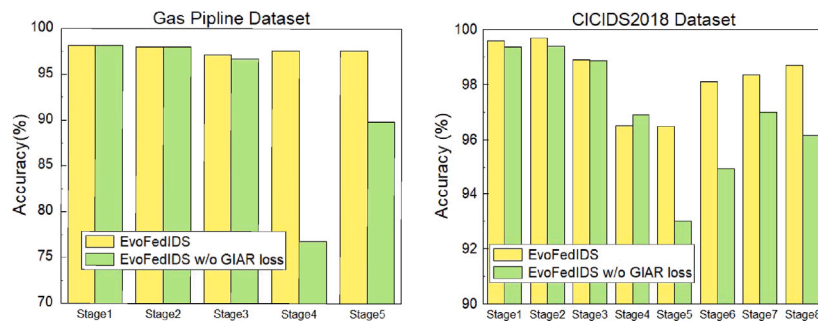


Fig. 13. Effect of the proposed GIAR loss on the performances.

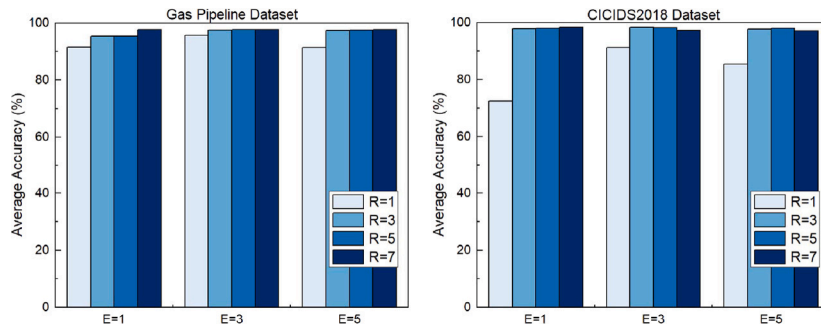


Fig. 14. Impact of E and R on the performances.

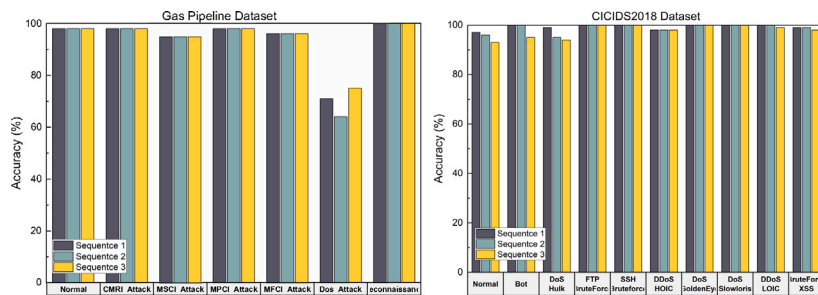


Fig. 15. The attack detection accuracies of EvoFedIDS under different task sequences.

the accuracies substantially drop to 20% in Stage 4 compared with EvoFedIDS. It proves the necessity of the proposed GIAR loss and the effectiveness in mitigating the negative impact of heterogeneity.

Impact of E and R : Observed from Fig. 14, EvoFedIDS achieves the lowest average accuracies with $R=1$, while the accuracies have improved significantly with $R=3$. It indicates that the increase of communication rounds promotes the knowledge sharing across the clients and then boosts the performances of EvoFedIDS. The increase of E has little effect on performance on gas pipeline dataset, since the small quantity of traffic samples do not require extensive local training. Thus, it is vital to choose the appropriate values of E and R for balancing between effectiveness and efficiency.

Sensitivity analysis of task execution sequence: In deploying EvoFedIDS within dynamic IIoT environments, it is crucial to consider how changes in network topology and device behavior affect model performance. Examining the CICIDS2018 dataset as a case study, we note that each packet sample contains information about network topology (source and target IP addresses). Depending on the implemented network topology, each type of attack corresponds to a specific scenario. For instance, in the FTP-BruteForce attack scenario, the attacker employs a Kali Linux system with an IP address of 172.31.70.4, targeting a victim machine running Ubuntu 14.0 with an IP address of 172.31.69.25. In our experiments, every IP address represents a

potential participating client. As the network topology and device behavior change, so do the participating clients, affecting the execution sequence of learning tasks. Herein, we investigate the impact of task execution sequences and provide the model performance results at the final stage under different task sequences. Specifically, we randomize the task order to construct different tasks sequences. Fig. 15 demonstrates that varying task execution sequences have a negligible impact on our performance. This demonstrates the resilience and capability of EvoFedIDS to maintain consistent performance despite evolving network conditions and device behaviors.

5.3. Efficiency analysis

5.3.1. Computation complexity

The computational complexity of EvoFedIDS is influenced by two primary factors: local continual representation learning and global knowledge aggregation. The process of updating the local embedding network model involves a complex operation with a time complexity of $O(E \cdot |\tilde{D}_i'| \cdot |\Theta_i'|)$. Regarding the global knowledge aggregation, the complexity is determined by the global aggregation of the local embedding network models, class prototypes, and local model importance factors. This aggregation complexity can be formalized as $O(|\tilde{\Theta}'| + |\mathcal{C}'| |\vec{\mu}_c'| + |\vec{\Gamma}'|)$, where $|\tilde{\Theta}'|$ represents the number of the local model parameters,

Table 6

The average local training time on gas pipeline dataset and CICIDS2018 dataset.

Dataset	Fedcarl	FCIL	Fedproto	EvoFedIDS
Gas Pipeline	0.1987	0.2510	0.5142	0.8269
CICIDS2018	4.4785	5.7092	4.7265	10.3763

Table 7

The uplink and downlink communication overhead analysis.

Method	Direction	Communication overhead
Fedproto	Uplink	$ \Theta'_i + C'_i \mu'_{i,c} $
	Downlink	$ \Theta' + C' \mu'_c $
Fedcarl/FCIL	Uplink	$ \Phi'_i $
	Downlink	$ \Phi' $
EvoFedIDS	Uplink	$ \Theta'_i + C'_i \mu'_{i,c} + \Gamma'_i $
	Downlink	$ \Theta' + C' \mu'_c + \Gamma' $

$|C'_i|$ is the number of categories have already encountered in this local client, $|\mu'_c|$ is the size of class prototype, $|\Gamma'|$ represents the number of local model importance factors.

Table 6 presents the average time spent on local training for various methods on both dataset. It is observed that EvoFedIDS requires slightly more time compared to other dynamic federated learning methods. Fedcarl, FCIL, and the proposed EvoFedIDS are all based on memory-replay methods. Nonetheless, whereas Fedcarl and FCIL employ a uniform sampling technique to update memory, EvoFedIDS adopts the prototype variance-based memory update strategy. This strategy necessitates additional computation of prototype variance, resulting in longer processing times. In comparison to Fedproto, EvoFedIDS introduces a global information-aware regularization loss to mitigate catastrophic forgetting, requiring extra computation of the local model importance factors $|\Gamma'|$. Despite these increased computational demands, the performances of EvoFedIDS are significantly better, rendering the tradeoff between performance and efficiency acceptable. Besides, all methods exhibit an average training time of less than 1 s on gas pipeline dataset, and the time consumptions for all methods are considerably higher for CICIDS2018 dataset. This can be attributed to the fact that CICIDS2018 dataset contains a significantly larger amount of data compared to gas pipeline dataset.

5.3.2. Communication complexity

During the uplink communication in EvoFedIDS, the local client uploads several components to the server. In addition to the current round's training of the local embedding network model Θ'_i , the client also needs to upload the current class prototype $\mu'_{i,c}$ and the local model importance factor Γ'_i . Therefore, the uplink communication overhead can be expressed as $|\Theta'_i| + C'_i |\mu'_{i,c}| + |\Gamma'_i|$. Subsequently, for downlink communication, the server combines the local network model, the class prototype, and local model importance factor uploaded by the local client to obtain the global embedding network model, global class prototype, and global model importance factor through the global aggregation. These global parameters are then broadcasted to all local clients. Hence, downlink communication overhead is $|\Theta'| + C' |\mu'_c| + |\Gamma'|$.

Table 7 also provides the uplink and downlink communication overheads of these methods. It shows that Fedproto does not need to transmit Γ'_i and has a lower communication overhead than EvoFedIDS. However, EvoFedIDS incorporates the global information-aware regularization loss, which enhances performance on heterogeneous traffic data. Additionally, the uplink communication overhead of Fedcarl and FCIL is $|\Phi'|$. As the number of neurons in the classifier network increases, the uplink communication overhead for Fedcarl and FCIL gradually increases as well.

6. Conclusion

In this work, we build the evolutionary distributed intrusion detection system based on federated continual representation learning, namely EvoFedIDS. It is designed to continually learn effective feature representations of new attacks from dynamic traffic. Concretely, we design the supervised contrastive loss and the global information-aware regularization loss to mitigate the catastrophic forgetting on the previously observed attacks and the negative influence of data heterogeneity. Besides, we propose the prototype variance-based memory update strategy to ensure the effective memory replay data. Extensive experiments verify the effectiveness and robustness of EvoFedIDS. Future work will focus on improving EvoFedIDS's adaptability to unknown attack types with advanced open-set recognition algorithms and extending its application across diverse IIoT systems.

CRedit authorship contribution statement

Zhao Zhang: Writing – original draft, Methodology. **Yong Zhang:** Funding acquisition. **Hao Li:** Investigation. **Shenbo Liu:** Validation. **Wei Chen:** Investigation. **Zhigang Zhang:** Formal analysis. **Lijun Tang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work is supported by the Postdoctoral Fellowship program of China Postdoctoral Science Foundation under Grant No. GZC20232241.

References

- Boppana, T.K., Bagade, P., 2023. GAN-AE: An unsupervised intrusion detection system for MQTT networks. *Eng. Appl. Artif. Intell.* 119, 105805.
- Cerasuolo, F., Bovenzi, G., Marescalco, C., et al., 2023. Adaptive intrusion detection systems: Class incremental learning for IoT emerging threats. In: 2023 IEEE International Conference on Big Data. BigData, IEEE, pp. 3547–3555.
- Chi, H.R., Wu, C.K., Huang, N.F., et al., 2022. A survey of network automation for industrial internet-of-things towards industry 5.0. *IEEE Trans. Ind. Inform.*
- Dong, J., Wang, L., Fang, Z., et al., 2022. Federated class-incremental learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 10164–10173.
- Ferrag, M.A., Maglaras, L., 2019. DeepCoin: A novel deep learning and blockchain-based energy exchange framework for smart grids. *IEEE Trans. Eng. Manage.* 67 (4), 1285–1297.
- Gyamfi, E., Jurcut, A.D., 2022. Novel online network intrusion detection system for industrial iot based on oi-svdd and as-elm. *IEEE Internet Things J.* 10 (5), 3827–3839.
- Hendryx, S.M., KC, D.R., Walls, B., et al., 2021. Federated reconnaissance: Efficient, distributed, class-incremental learning. *arXiv preprint arXiv:2109.00150*.
- Idrissi, M.J., Alami, H., El Mahdaoui, A., et al., 2023. Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems. *Expert Syst. Appl.* 234, 121000.
- Karn, R.R., Kudva, P., Elfadel, I.M., 2021. Learning without forgetting: A new framework for network cyber security threat detection. *IEEE Access* 9, 137042–137062.
- Khosla, P., Teterwak, P., Wang, C., et al., 2020. Supervised contrastive learning. *Adv. Neural Inf. Process. Syst.* 33, 18661–18673.
- Li, B., Wu, Y., Song, J., et al., 2020. DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems. *IEEE Trans. Ind. Inform.* 17 (8), 5615–5624.
- Ling, Z., Hao, Z.J., 2022a. An intrusion detection system based on normalized mutual information antibodies feature selection and adaptive quantum artificial immune system. *Int. J. Semant. Web Inf. Syst. (IJSWIS)* 18 (1), 1–25.

- Ling, Z., Hao, Z.J., 2022b. Intrusion detection using normalized mutual information feature selection and parallel quantum genetic algorithm. *Int. J. Semant. Web Inf. Syst. (IJSWIS)* 18 (1), 1–24.
- Morris, T., Gao, W., 2014. Industrial control system traffic data sets for intrusion detection research. In: *Critical Infrastructure Protection VIII: 8th IFIP WG 11.10 International Conference, ICCIP 2014, Arlington, VA, USA, March (2014) 17-19, Revised Selected Papers 8*. Springer Berlin Heidelberg, pp. 65–78.
- Prasath, S., Sethi, K., Mohanty, D., et al., 2022. Analysis of continual learning models for intrusion detection system. *IEEE Access* 10, 121444–121464.
- Salim, M.M., El Azzaoui, A., Deng, X., et al., 2024. FL-CTIF: A federated learning based CTI framework based on information fusion for secure IIoT. *Inf. Fusion* 102, 102074.
- Shenaj, D., Toldo, M., Rigon, A., et al., 2023. Asynchronous federated continual learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 5054–5062.
- Song, J., Li, B., Wu, Y., et al., 2020. ReAL: a new ResNet-ALSTM based intrusion detection system for the internet of energy. In: *2020 IEEE 45th Conference on Local Computer Networks. LCN, IEEE*, pp. 491–496.
- Srivastava, S., Yaqub, M., Nandakumar, K., 2023. Lifelong learning of task-parameter relationships for knowledge transfer. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 2524–2533.
- Tahir, B., Jolfaei, A., Tariq, M., 2021. Experience-driven attack design and federated-learning-based intrusion detection in industry 4.0. *IEEE Trans. Ind. Inform.* 18 (9), 6398–6405.
- Wu, Y., Dai, H.N., Tang, H., 2021. Graph neural networks for anomaly detection in industrial internet of things. *IEEE Internet Things J.* 9 (12), 9214–9231.
- Xu, X., Zhang, X., Zhang, Q., et al., 2024. Advancing malware detection in network traffic with self-paced class incremental learning. *IEEE Internet Things J.*
- Zenke, F., Poole, B., Ganguli, S., 2017. Continual learning through synaptic intelligence. In: *International Conference on Machine Learning*. PMLR, pp. 3987–3995.
- Zhang, X., Dong, S., Chen, J., et al., 2022a. Deep class-incremental learning from decentralized data. *IEEE Trans. Neural Netw. Learn. Syst.*
- Zhang, H., Liu, B., Wu, H., 2021. Smart grid cyber-physical attack and defense: A review. *IEEE Access* 9, 29641–29659.
- Zhang, Z., Zhang, Y., Guo, D., et al., 2022b. SecFedNIDS: Robust defense for poisoning attack against federated learning-based network intrusion detection system. *Future Gener. Comput. Syst.* 134, 154–169.
- Zhang, Z., Zhang, Y., Guo, D., et al., 2023. Communication-efficient federated continual learning for distributed learning system with non-IID data. *Sci. China Inf. Sci.* 66 (2), 122102.